

אתר "מודעה"

מסמך תכנון

3	תיאור כללי
3	מינוחים בשימוש במסמך
3	טכנולוגיות בשימוש המערכת
3	העלאת המערכת
3	נקודות כניסה למערכת
4	בסיס הנתונים
4	גישה לבסיס הנתונים
4	הטבלאות בבסיס הנתונים
5	ERD
6	תרשים כללי של המערכת
7	ספריות JAVA עיקריות שבשימוש
8	JSPs ה
8	JSPs לטיפול במנהלנים
10	JSPs לטיפול במשתמשי הקצה
11	מחלקות ה JAVA
11	UML – קשרים ומתודות עיקריות
12	חבילת ה Servlet
12	מחלקת ה StartupServlet
12	מחלקת ה UserXmlProcessor
13	מחלקת ה XmlProcessor
13	חבילת ה Users
13	מחלקת ה User
13	מחלקת ה UserDBEnum
14	מחלקת ה UserImpl
14	חבילת ה Assets
14	מחלקת ה Asset
15	מחלקת ה AssetDBEnum
15	מחלקת ה AssetImpl
15	מחלקת ה AssetSearchCriteria
16	חבילת ה Data
16	מחלקת ה ObjectData
16	מחלקת ה ObjectTypesDBEnum
16	מחלקת ה ObjectDataImpl
17	חבילת ה Address
17	מחלקת ה City
17	מחלקת ה Street
18	מחלקת ה AddressImpl
18	חבילת ה XML
18	מחלקת ה DomWrapper
19	חבילת ה Exceptions
19	מחלקת ה UserAlreadyExistException
19	מחלקת ה UserNotFoundException
19	חבילת ה SQL
19	מחלקת ה AggregateResultProcessor
19	מחלקת ה DBUtils
19	חבילת ה Common
19	מחלקת ה BaseForm
19	מחלקת ה CacheManager
19	מחלקת ה MinMax
20	מחלקת ה Sort
20	מחלקת ה Utils

תיאור כללי

מסמך זה בא לתאר את מבנה המערכת שמרכיבה את אתר "מודעה" ואת הטכנולוגיות שבשימוש.

מינוחים בשימוש במסמך

מינהלן – משתמש במערכת מסוג מנהל אשר מוסיף ומעדכן מודעות במערכת.
משתמש קצה – משתמש במערכת שהוא קהל היעד של המערכת אשר מתחבר כדי לחפש בה מודעות.

טכנולוגיות בשימוש המערכת

אתר האינטרנט "מודעה" משלב בתוכו מספר טכנולוגיות:

- MySQL Server – מכיל את טבלאות הנתונים של המערכת.
 - JDBC – מאפשר גישה לבסיס הנתונים.
 - JAVA classes, Servlets – קוד JAVA שרץ בצד השרת.
 - JSP – עמודי HTML משולבים בקוד JAVA ומחוללים עמודים HTML לצד הלקוח.
 - Struts Taglib – מרכיב Taglib מספק קבוצה של ספריות עם תגי JSP מותאמים אישית לסייע למפתחים ליצור יישומים מבוססי טופס בצורה אינטראקטיבית. יש תגיות כדי לעזור עם כל דבר, החל מלהציג הודעות שגיאה ועד יצירת תגים מקוננים לשימוש חוזר.
 - Tomcat + JRE – שרת WEB אשר מריץ את ה Servlets וקבצי ה JSP.
 - DHTML ו Java Script – מאפשר עיצוב גמיש של עמודי ה HTML ואינטראקציה עם המשתמשים
 - AJAX – ראשי תיבות של XMLHttpRequest And JavaScript Asynchronous, היא טכניקה ליצירת יישומי דפדפן אינטראקטיביים המבוססים על קוד המורץ במסגרת דף HTML בודד, ולא כיישום מרובה דפים, כמקובל בסביבת הווב. מטרתה העיקרית של הטכניקה היא שיפור חוויית המשתמש והאצת מהירות הטעינה של דפי האינטרנט, מאחר שהיא מאפשרת לעדכן רק חלקים מבוקשים בדף האינטרנט, ללא צורך לטעון את הדף כולו מחדש במחשבו של המשתמש.
- בהמשך המסמך יתוארו הטכנולוגיות העיקריות ואופן השימוש בהן וכמו כן תתוארנה המחלקות השונות והקשרים ביניהן.

העלאת המערכת

בכדי לעבוד עם המערכת יש להעלות את שרת ה MySQL ואת שרת ה Tomcat. קובץ בשם web.xml מכיל הגדרות שונות אשר משמשות את ה Tomcat. אחת מהן מגדירה ל Tomcat להריץ בזמן העלייה, Servlet בשם StartupServlet. זה אחראי לאיתחולים שונים של המערכת כאשר החשוב מביניהם הוא איתחול החיבור מול בסיס הנתונים. המאפיינים של בסיס הנתונים מוגדרים בקובץ modaa.properties אשר בין היתר מכיל נתונים לגבי איתחול של ספרייה אחרת שמטפלת בלוגים של המערכת.

נקודות כניסה למערכת

הגישה למערכת מתבצעת ב HTTP מול שרת ה Tomcat אשר מאזין לפורט 8080 במחשב המקומי. הגישה היא בעזרת הלינקים הבאים:
<http://localhost:8080/modaa> - חיבור למשתמשי קצה.
<http://localhost:8080/modaa/admin> - חיבור למינהלנים.
בשני המקרים מורץ בצד השרת הקובץ index.jsp (כל לינק והקובץ שלו) שזוהי למעשה נקודת הכניסה שמתחילה את הטיפול במשתמשים.

בסיס הנתונים

במערכת זו ישנו בסיס נתונים השומר בתוכו הן נתונים סטטיים המוכנים מראש והן נתונים דינאמיים אשר נקלטים מהמשתמשים.
בסיס הנתונים ממומש בעזרת MySQL Server, המאזין על TCP/IP פורט ברירת המחדל (3306), והגישה אליו נעשית בעזרת התקן של My SQL JDBC.

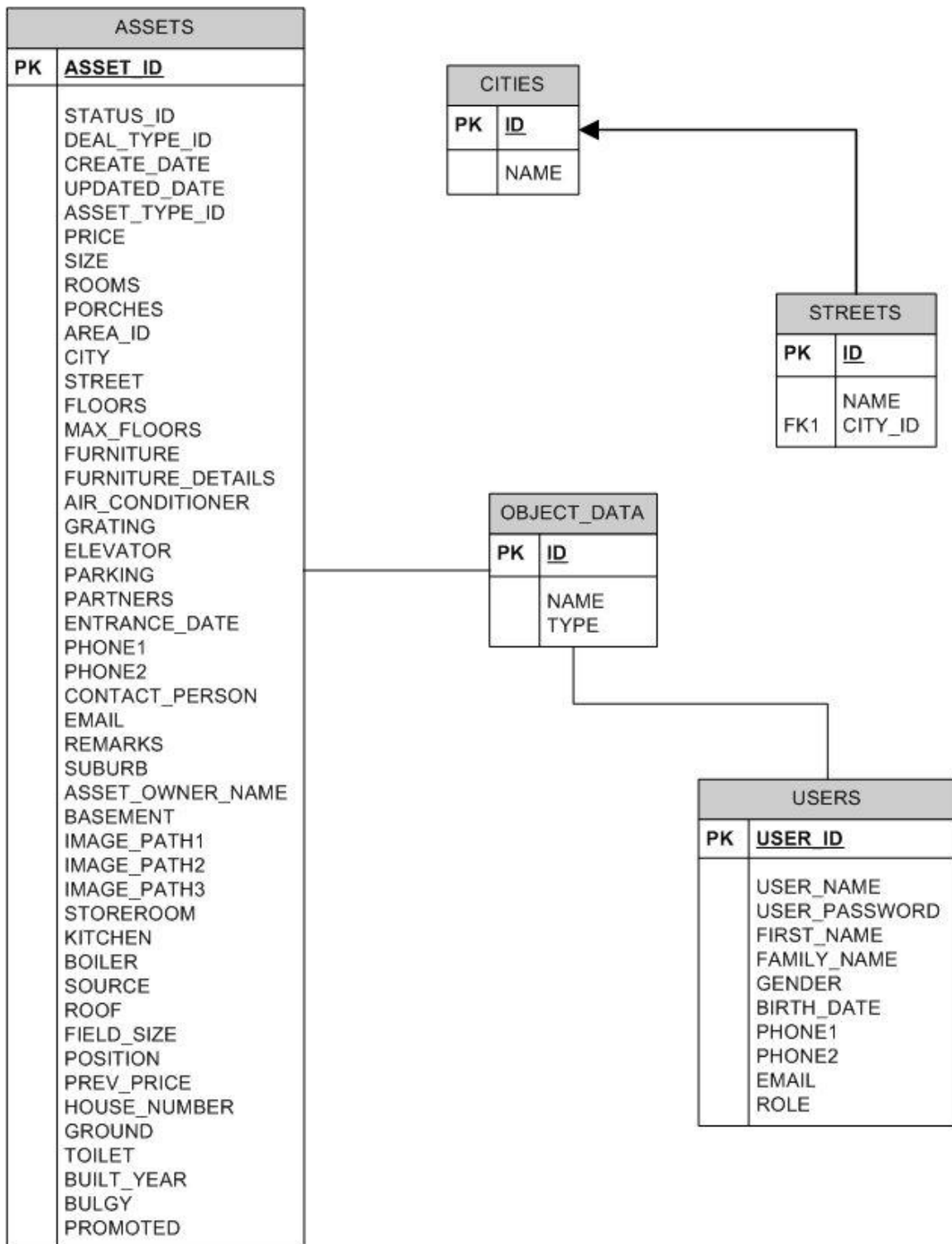
גישה ל בסיס הנתונים

הגישה לבסיס הנתונים מתבצעת ע"י הרצה של ה My SQL Server והגישה אליו מתבצעת בשימוש במחלקת Java בשם util.sql.ConnectionManager, אשר מאותחל לעבוד בעזרת התקן של My SQL JDBC, ובמחלקות נוספות בספריית sqlFramework.jar אשר מאפשרות הרצת שאילות וקבלת התשובות מול בסיס הנתונים.

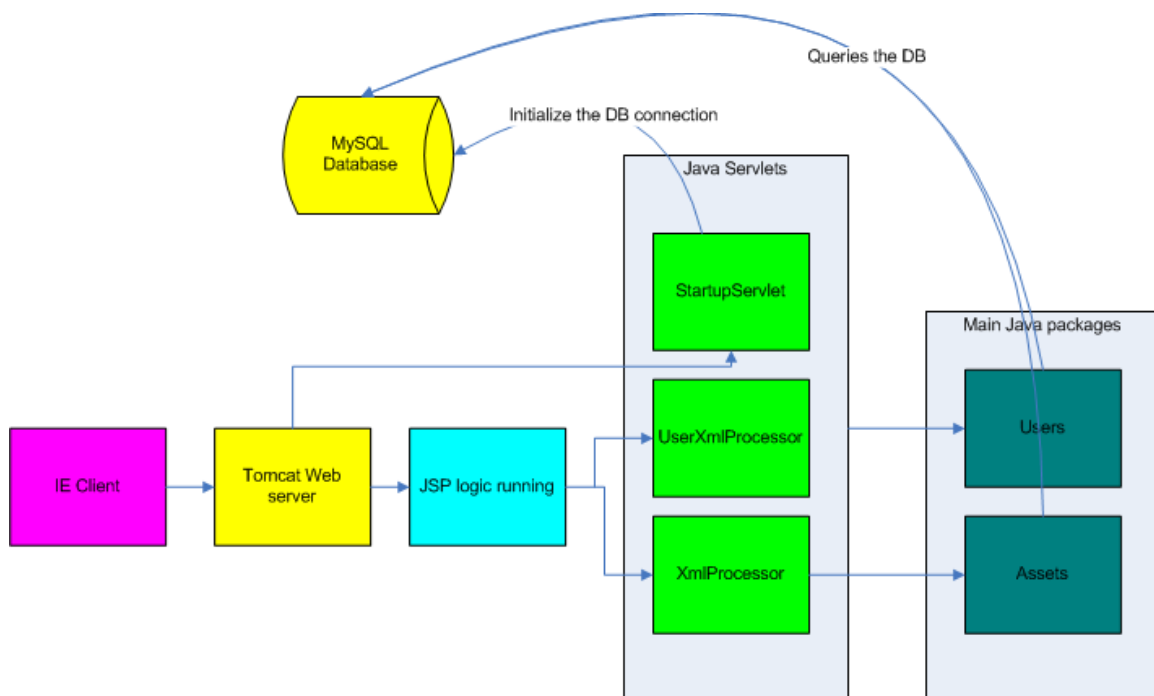
הטבלאות ב בסיס הנתונים

מאגר הנתונים נקרא modaa ומכיל את הטבלאות הבאות:

שם טבלה	הגדרת הטבלה
Assets	מכילה את המודעות המתפרסמות באתר. מנהלנים מעדכנים את הטבלה הזו, ומשתמשי קצה מחפשים בה נתונים. הטבלה מכילה נתונים שונים לגבי הנכס, ומכילה "הצבעות" לטבלאות שמופיעות בהמשך, בעזרת שימוש בשדה בשם ID המשמש כמפתח של הטבלאות. מפתח: ה ID של הרשומה.
cities	רשימת ערים בישראל. מפתח: ה ID של הרשומה.
Streets	רשימת רחובות בערים בישראל. לכל רשומה יש את שם הרחוב והצבעה לעיר לה הוא שייך. מפתח: ה ID של הרשומה בחיבור עם ה ID של העיר.
object_data	מכילה אוסף של נתונים כגון: מכירה/השכרה, מספר קומות בבניין, סוג הנכס (דירה/דירת גג...). הרעיון הוא שיש סוגים של נתונים, ושכל סוג נתונים, יש מספר נתונים. מפתח: ה ID של הרשומה בחיבור עם ה ID של סוג הנתון.
Users	פרטי המשתמשים במערכת, כולל שם המשתמש והסיסמה כאשר יש שני סוגי הרשות משתמשים: מנהלן ומשתמש קצה. מפתח: ה ID של הרשומה.



תרשים כללי של המערכת



ספריות JAVA עיקריות שבשימוש

שם ספרייה	תאור	JAR-ים
Struts 1.3	מרכיב Taglib מספק קבוצה של ספריות עם תגי JSP מותאמים אישית לסייע למפתחים ליצור יישומים מבוססי טופס בצורה אינטראקטיבית. יש תגיות כדי לעזור עם כל דבר, החל מלהציג הודעות שגיאה ועד יצירת תגים מקוננים לשימוש חוזר.	antlr-2.7.2.jar bsf-2.3.0.jar commons-beanutils-1.8.0.jar commons-chain-1.2.jar commons-digester-1.8.jar commons-fileupload-1.1.1.jar commons-io-1.1.jar commons-logging-1.0.4.jar commons-validator-1.3.1.jar jstl-1.1.2.jar oro-2.0.8.jar standard-1.0.6.jar struts-core-1.3.10.jar struts-el-1.3.10.jar struts-extras-1.3.10.jar struts-faces-1.3.10.jar struts-mailreader-dao-1.3.10.jar struts-scripting-1.3.10.jar struts-taglib-1.3.10.jar struts-tiles-1.3.10.jar
MySQL JDBC driver	ספרייה שמאפשרת קישוריות לשרת MySQL	mysql-connector-java-3.1.10-bin.jar
XStream	ספרייה שמאפשרת סיריאליזציה של אובייקט ל XML ובחזרה.	xstream-1.3.jar jdom-1.0.jar dom4j-1.6.1.jar joda-time-1.0.jar stax-1.1.1-dev.jar stax-api-1.0.jar xml-writer-0.2.jar xom-1.0b3.jar xpp3-1.1.3.4d_b4_min.jar
LOG4j	ספרייה המאפשרת כיבוי והדלקה של לוגים שהמערכת מייצרת ע"י הגדרה של קובץ קונפיגורציה וללא צורך להוריד ולהעלות את התוכנית.	log4j-1.2.15.jar commons-collections-3.1.jar commons-lang-2.4.jar
SQL Framework	ספרייה המספקת מסגרת לעבודה עם SQL.	sqlFramework.jar

ה JSPs

במערכת ישנם JSPs שונים לצורך טיפול בבקשות משתמשים. ה JSPs מחולקים בגדול לשניים: טיפול במנהלים, וטיפול במשתמשי קצה. בנוסף, ישנם מספק קבצי JSP בספריה הראשית אשר משמשים לתצוגה.

JSPs לטיפול במנהלים

הטיפול במנהלים נמצא תחת ספריית modaa/admin ולהלן הסבר על כל קובץ:

שם קובץ JSP	תאור הקובץ
Index.jsp	זהו הקובץ הראשי בספרייה אשר ה Tomcat יריץ אוטומטית בגלישה לספרייה. מטרת הקובץ היא לטפל בתהליך הכניסה של מנהלים לאתר ע"י קליטת שם משתמש וסיסמה ושליחתם לשרת, בעזרת בקשת XML לצורך וידוא הפרטים. - פונקצית doLoad נקראת בטעינת העמוד ובודקת עם המשתמש כבר נרשם, ואם כן, אז היא מעבירה אותו אוטומטית לעמוד הבא – newAssest.jsp. - פונקצית doLogin נקראת כאשר המשתמש ממלא את פרטיו ומקיש על "כניסה". הפונקציה יוצרת בקשת XML ושולחת אותה לשרת לצורך וידוא פרטים, ומעבירה כפרמטר את הפונקציה הבאה: login. - פונקצית login נקראת כאשר חוזרת תשובת ה XML מהשרת. במקרה של הצלחה, המנהל יועבר אוטומטית למסך הבא – newAssets.jsp. במקרה של כישלון, תוצג הודעת שגיאה.
newAssets.jsp	מטרת הקובץ היא להתחיל תהליך על הוספת מודעה חדשה, או עדכון מודעה קיימת בעזרת קליטה של אפס או יותר מהפרטים הבאים: מספר טלפון, רחוב, מספר בית. בנוסף, מתחתית העמוד יש הפנייה למסך חיפוש מודעה לצורך עדכונה – search.jsp. - פונקצית doSubmit נקראת כאשר המשתמש מקיש על "הכנס" או מקיש על ENTER. במידה ואחד מהשדות מולאו ע"י המשתמש, היא בונה בקשת XML בעזרת הפונקציה getSearchCriteriaXml ושולחת אותה לשרת לצורך בדיקה אם קיימת מודעה עם הפרטים הללו או לא. התשובה ל XML תחזור לפונקציה handleResult. במידה והמשתמש לא מילא פרטים, היא תקרא ישירות לפונקציה redirect לצורך מעבר ל asset.jsp לצורך הגדרת נכס חדש. - פונקצית handleResult בודקת אם חזרה מהשרת תגובה שיש כזו רשומה. אם יש, המשתמש נשאל אם הוא רוצה לערוך את הרשומה הקיימת או להוסיף חדשה. אם אין רשומה בשרת, המשתמש מועבר לעמוד הוספת רשומה חדשה. הנ"ל נעשה בעזרת פונקצית redirect. - פונקצית redirect מעבירה את המשתמש למסך הבא – assets.jsp כאשר המסך יכול להיות עריכת נכס קיים, או הוספת נכס חדש. ההעברה מתבצעת ע"י יצירת טופס נסתר ו submit שלו בצורה אוטומטית עם פרטי הנכס ומידה ויש כאלו.

שם קובץ JSP	תאור הקובץ
asset.jsp	הקובץ מטפל בהוספה, עריכה או מחיקה של נכס. מגיעים אל הקובץ מ <code>newAssets.jsp</code> או מ <code>search.jsp</code>. הקובץ קורא את הפרמטר "id" אשר מייצג מספר נכס. במידה המספר הוא 0, הקובץ יהיה במצב של נכס חדש, אחרת יהיה במצב של עריכת הנכס שזה ה "id" שלו. המסך מכיל שדות שונים לצורך קליטת פרטי הנכס. במסך יש כפתור לצורך עדכון הנכס, ובמצב של עריכה, יהיה גם כפתור מחיקה. בסוף העדכון או המחיקה, המשתמש יוכל לבחור אם להישאר בעמוד הנוכחי ולעדכן את הרשומה שוב, או לעבור ל <code>newAsset.jsp</code> לצורך עדכון או הוספה של נכס נוסף. - פונקציית <code>doSubmit</code> נקראת כאשר המשתמש מקיש על "שמור" והיא מייצרת בקשת XML לצורך עדכון הנכס במאגר הנתונים. תשובת ה XML תחזור לפונקציית <code>handleSubmit</code>. - פונקציית <code>handleSubmit</code> נקראת לטיפול בתשובת ה XML בסוף עדכון הנכס במאגר הנתונים. - פונקציית <code>doDelete</code> נקראת כאשר המשתמש מקיש על "מחק" והיא מייצרת בקשת XML לצורך מחיקת הנכס לפי ה "id" שלו. תשובת ה XML תחזור לפונקציית <code>handleDeleteResponse</code>. - פונקציית <code>handleDeleteResponse</code> נקראת כאשר התשובה שהנכס נמחק מתקבלת, ומעבירה לעמוד ה <code>newAsset.jsp</code>.
search.jsp	הקובץ מציג בפני המנהלן קטגוריות לחיפוש על מאגר הנכסים ומציג את התוצאות עם אפשרות לראות את פרטי הנכס ולערוך אותו. הקובץ הזה מכיל למעשה את שני המסכים (חיפוש ותוצאות חיפוש) כאשר ברגע נתון רק אחד מהם מוצג. ההחלפה מתבצעת בעזרת DHTML ו JS בהתאם לבחירות המשתמש. מסך התוצאות כולל אפשרויות מיון של התוצאות לפי הקטגוריות השונות. בנוסף, במידה ויש יותר ממסך אחד של תוצאות, ניתן לדפדף בין המסכים. - פונקציית <code>doSubmit</code> נקראת בכדי לחפש בבסיס הנתונים את הנכסים לפי הקריטריונים שהמשתמש בחר. הפונקציית מייצרת בקשת XML ושולחת אותה לטיפול בשרת. תשובת ה XML תחזור לפונקציית <code>handleResult</code>. בנוסף, הפונקציית יודעת לבצע את השאילתא ממוינת, או להתחיל מנקודה מסוימת בתוצאות החיפוש, לשימוש הפונקציות <code>doSort</code> ו <code>navigate</code>. - פונקציית <code>handleResult</code> - פונקציית <code>doSort</code> קוראת מחדש ל <code>doSubmit</code> עם פרמטרים למיון בכדי להציג את התצוגה ממוינת לפי העמודה עליה הקיש המשתמש לצורך מיון. - פונקציית <code>navigate</code> נקראת כאשר המשתמש לוחץ על "לקודמים" או "לבאים" בכדי לראות תוצאות חיפוש נוספות במקרה שמספר תוצאות החיפוש גדול מהמסך. - פונקציית <code>showHideMessage</code> פותחת בחלון חדש את פרטי המודעה המלאים. העמוד החדש מופנה לקובץ <code>assets/showAssetTemplate.asp</code> שמתואר בהמשך בחלק ה JSPs לטיפול במשתמשי קצה.

JSPs לטיפול במשתמשי הקצה

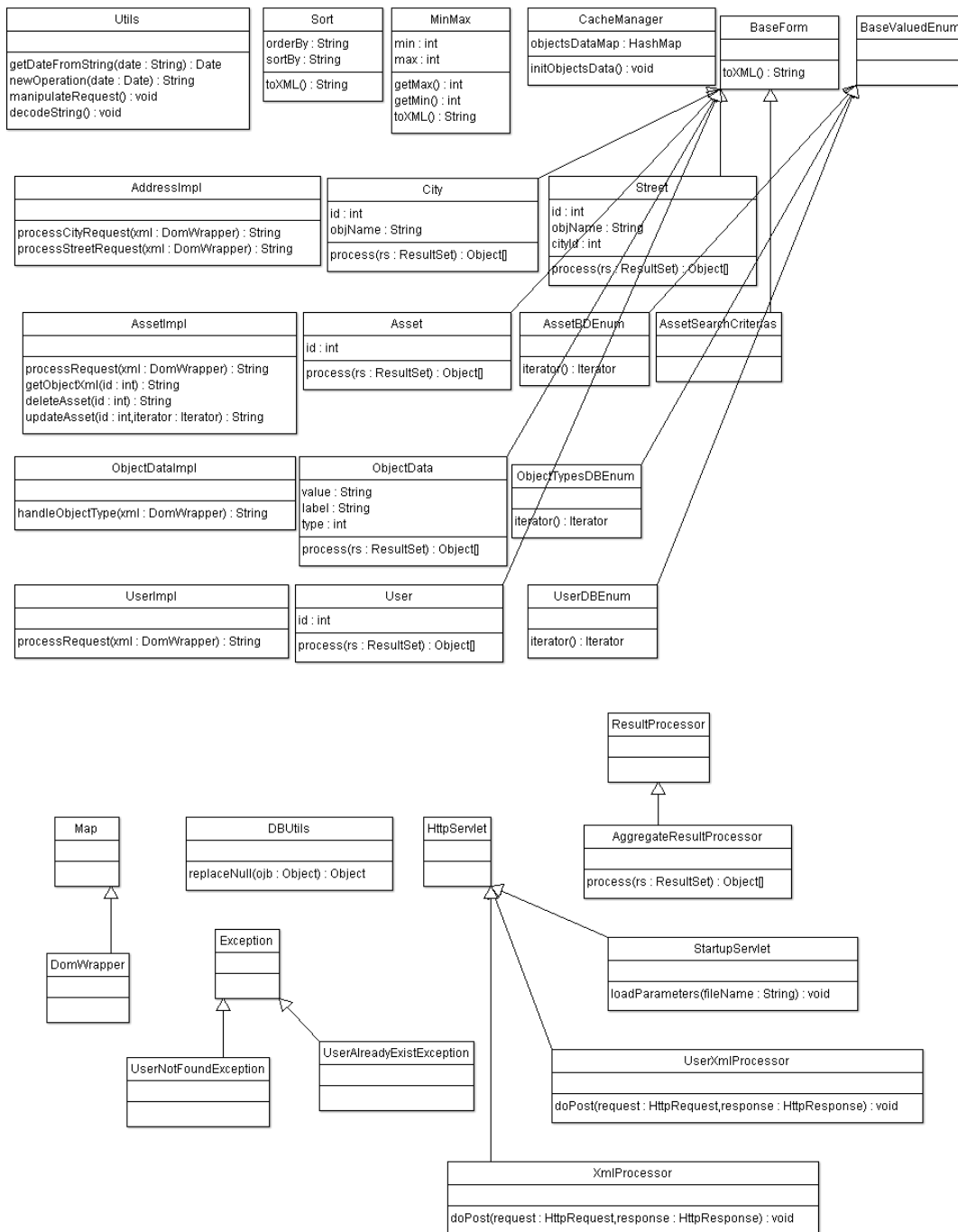
הטיפול במשתמשי הקצה נמצא תחת ספריית modaa/assets ו modaa/users ולהלן הסבר על כל קובץ:

שם קובץ JSP	תאור הקובץ
search.jsp	הקובץ מציג בפני משתמש הקצה קטגוריות לחיפוש על מאגר הנכסים ומציג את התוצאות עם אפשרות לראות את פרטי הנכס, במידה והמשתמש נרשם לאתר, או אפשרות להירשם לאתר. הקובץ הזה מכיל למעשה את שני המסכים (חיפוש ותוצאות חיפוש) כאשר ברגע נתון רק אחד מהם מוצג. ההחלפה מתבצעת בעזרת DHTML ו JS בהתאם לבחירות המשתמש. מסך התוצאות כולל אפשרויות מיון של התוצאות לפי הקטגוריות השונות. בנוסף, במידה ויש יותר ממסך אחד של תוצאות, ניתן לדפדף בין המסכים. בנוסף, בצד ימין של המסך, מוצג חלון רישום עבור המשתמש. - פונקציית doSubmit נקראת בכדי לחפש בבסיס הנתונים את הנכסים לפי הקריטריונים שהמשתמש בחר. הפונקציה מייצרת בקשת XML ושולחת אותה לטיפול בשרת. תשובת ה XML תחזור לפונקציה handleResult. בנוסף, הפונקציה יודעת לבצע את השאילתא ממוינת, או להתחיל מנקודה מסוימת בתוצאות החיפוש, לשימוש הפונקציות doSort ו navigate. - פונקציית handleResult - פונקציית doSort קוראת מחדש ל doSubmit עם פרמטרים למיון בכדי להציג את התצוגה ממוינת לפי העמודה עליה הקיש המשתמש לצורך מיון. - פונקציית navigate נקראת כאשר המשתמש לוחץ על "לקודמים" או "לבאים" בכדי לראות תוצאות חיפוש נוספות במקרה שמספר תוצאות החיפוש גדול מהמסך. - פונקציית showHideMessage פותחת בחלון חדש את פרטי המודעה המלאים. העמוד החדש מופנה לקובץ admin/showAssetTemplate.asp שמתואר בהמשך בחלק ה JSPs לטיפול במשתמשי קצה. - פונקציית doLogin נקראת כאשר המשתמש ממלא את פרטיו ומקיש על "כניסה". הפונקציה יוצרת בקשת XML ושולחת אותה לשרת לצורך וידוא פרטים.
showAssetTemplate.jsp	המסך הזה נקרא מ admin/search.jsp ומ asset/search.jsp ומציג את הנתונים המלאים של הנכס. - פונקציית doLoad שולחת בקשת XML לשרת לצורך קבלת הנתונים המלאים של הנכס. כאשר התשובה חוזרת היא קוראת לפונקציית parseParams לצורך קריאת התשובה ומילוי המסך בנתונים.
user.jsp	הקובץ מטפל ברישום של משתמשי קצה חדשים למערכת. למשתמש מוצג מסך שבו עליו למלא את פרטיו הכוללים שם משתמש, סיסמה ועוד. בסיום, הבקשה נשלחת לשרת לצורך שמירת הנתונים. - פונקציית doSubmit נקראת כאשר המשתמש לוחץ על "שמור" לצורך רישום המשתמש החדש במאגר הנתונים. הפונקציה יוצרת בקשת XML לשרת והתשובה תטופל בפונקציה handleSubmit. - פונקציית handleSubmit מציגה למשתמש הודעת כישלון במידה והמשתמש כבר קיים, או הצלחה אם המשתמש החדש נרשם במאגר הנתונים.

מחלקות ה JAVA

להלן תאור של מחלקות ה Java שבתוכנית, לחבילות שונות, כאשר חבילה ולכל מחלקה יש תיאור כללי, ובכל מחלקה ישנו תאור של המתודות העיקריות.

UML – קשרים ומתודות עיקריות



חבילת ה Servlet

חבילת זו מכילה מספר מחלקות אשר מוגדרות בשרת ה Tomcat בקובץ ה web.xml ומגדירות את ה URL בשרת אשר בעזרתו ניתן להגיע למחלקות האלו ולהריץ את הקוד שלהן.

מחלקת ה StartupServlet

מחלקה זו מורצת בזמן טעינת ה Tomcat ותפקידה העיקרי הוא להתחבר לבסיס הנתונים.

מתודות	הסבר
Init	נקראת בזמן עליית ה Tomcat, קוראת את קובץ ההגדרות modaa.properties וקוראת ל loadParameters
loadParameters	פונקציה מעטפת אשר קוראת את ההגדרות מהקובץ, וקוראת ל initDBConnection
initDBConnection	מאתחלת את החיבור לבסיס הנתונים לפי ההגדרות בקובץ.

מחלקת ה UserXmlProcessor

מחלקה זו מורצת כאשר המשתמש בוחר לעשות פעולות הקשורות למשתמש, כגון: רישום משתמש חדש למערכת וכניסה של משתמש קיים למערכת. הגישה אליה היא בעזרת הלינק <http://localhost:8080/modaa/UserXmlProcessor> אשר מוגדר בקובץ ה web.xml.

מתודות	הסבר
doPost	נקראת כאשר קוד ה JSP שולח מיצר בקשת XML ושולח אותה אל האובייקט הזה לטיפול. בעזרת מחלקת Utils, וקריאה ל manipulateRequest, מתורגמת בקשת ה HTTP לבקשת XML ובעזרת מחלקת DomWrapper, נוצר אובייקט אשר מאפשר גישה לפרמטרים בבקשת ה XML. הבקשה מועברת לטיפול ע"י מחלקת UserImpl בעזרת קריאה ל processRequest, ותשובת ה XML שמתקבלת, מוחזרת כתשובה ל JSP.

מחלקת ה *XmlProcessor*

מחלקה זו מורצת כאשר המשתמש בוחר בכל הפעולות אשר אינן קשורות בטיפול במשתמשים, כגון: חיפוש במאגר הנתונים והוספת נכס חדש. הגישה אליה היא בעזרת הלינק <http://localhost:8080/modaa/XmlProcessor> אשר מוגדר בקובץ `web.xml`.

מתודות	הסבר
<code>doPost</code>	נקראת כאשר קוד ה JSP מייצר בקשת XML ושולח אותה אל האובייקט הזה לטיפול. בעזרת מחלקת <code>Utils</code> , וקריאה ל <code>manipulateRequest</code> , מתורגמת בקשת ה HTTP לבקשת XML. המתודה <code>handleRequest</code> נקראת להמשך טיפול ותשובת ה XML שמתקבלת, מוחזרת כתשובה ל JSP.
<code>handleRequest</code>	בעזרת מחלקת <code>DomWrapper</code> , נוצר אובייקט אשר מאפשר גישה לפרמטרים בבקשת ה XML. המתודה מחפשת, לפי שם האובייקט בבקשת ה XML, את האובייקט המתאים לטיפול בבקשה זו וקריאה למתודה המתאימה. האובייקטים השונים לטיפול בבקשות הם: <code>AssetImpl</code> , <code>ObjectDataImpl</code> , <code>AddressImpl</code> . מחזירה תשובת XML.

חבילת ה *Users*

חבילה זו מכילה מספר מחלקות אשר מטפלות במשתמשים.

מחלקת ה *User*

מחלקה זו מייצגת משתמש בטבלת `users` בבסיס הנתונים ומספקת מתודות לקבלת וקביעת פרמטרים שונים של משתמש, כגון: שם המשתמש, סיסמה, תאריך לידה וכדומה. המחלקה בשימוש ע"י מחלקת `UserImpl` שתואר בהמשך. מספר דוגמאות למתודות שהמחלקה מספקת מתוארות להלן. המחלקה יורשת ממחלקת `BaseForm`.

מתודות	הסבר
<code>process</code>	בשל הירושה מ <code>BaseForm</code> מממש את המתודה הזו אשר מקבלת תוצאה של שאילתא בבסיס הנתונים, ומייצרת מערך של אובייקטים מסוג <code>User</code> שמייצגים את הנתונים מהשאילתא.
<code>fromXML</code>	מייצרת אובייקט של <code>User</code> בהינתן <code>User</code> בפורמט XML.
<code>toObjects</code>	מחזירה מערך של אובייקטים מסוג <code>Object</code> כאשר כל אחד מייצג פרמטר אחד של ה <code>User</code> . זה משמש לצורך עדכון בסיס הנתונים.
<code>getId/setId</code>	מחזירה/מעדכנת את המספר המזהה של המשתמש במערכת.
<code>getRole/setRole</code>	מחזירה/מעדכנת את תפקידו של המשתמש: משתמש קצה או מנהלן.
<code>getUserName/setUserName</code>	מחזירה/מעדכנת את שם המשתמש.
	ישנן מתודות נוספות לצורך קבלה ועדכון.

מחלקת ה *UserDBEnum*

מחלקה זו מקשרת בין מחלקת `User` לבין הרשומה בבסיס הנתונים. המחלקה מכילה את שמות השדות המייצגים `User` בבסיס הנתונים. המחלקה יורשת ממחלקת `BaseValuedEnum`.

מחלקת ה UserImpl

מחלקה זו מטפלת בבקשות של רישום משתמש חדש וכניסה של משתמש קיים. הבקשה מועברת מ UserXmlProcessor Servlet והמחלקה מנהלת את הרישום ואת הקריאה מול בסיס הנתונים.

מתודות	הסבר
processRequest	נקראת מ UserXmlProcessor ובודקת באיזה פעולה מדובר: קבלת משתמש קיים, כניסה למערכת, או הוספה ועדכון משתמש. בהתאם לכל אחת מהפעולות, תקראנה המתודות המתאימות שמפורטות להלן, ותוחזר תשובה בפורמט XML.
getObjectXml	מקבלת את ה "id" של המשתמש כפרמטר ומחזירה מחרוזת בפורמט XML המייצגת את המשתמש הנ"ל, במידה והוא נמצא בבסיס הנתונים.
login	בודקת אם יש בטבלת users משתמש ששם המשתמש והסיסמה שלו תואמים לפרמטרים מבקשת ה XML. בנוסף, מתבצעת בדיקה שתפקיד המשתמש (משתמש קצה או מנהלן) תואמים את הבקשה. מחזירה את המשתמש במידה ונמצא. במידה והבדיקה נכשלה, זורקת exception מסוג UserNotFoundException.
createUser	יוצרת משתמש חדש בטבלת users. במידה וכבר קיים משתמש עם אותו שם משתמש, זורקת exception מסוג UserAlreadyExistException.
updateUser	מעדכנת משתמש קיים בטבלת users ולא מחזירה ערך.

חבילת ה Assets

חבילה זו מכילה מספר מחלקות אשר מטפלות בנכסים.

מחלקת ה Asset

מחלקה זו מייצגת נכס בטבלת assets בבסיס הנתונים ומספקת מתודות לקבלת וקביעת פרמטרים שונים של נכס, כגון: מחיר, כתובת, מספר חדרים וכדומה. המחלקה בשימוש ע"י מחלקת AssetImpl שתתואר בהמשך. מספר דוגמאות למתודות שהמחלקה מספקת מתוארות להלן. המחלקה יורשת ממחלקת BaseForm.

מתודות	הסבר
process	בשל הירושה מ BaseForm מממשת את המתודה הזו אשר מקבלת תוצאה של שאילתא בבסיס הנתונים, ומייצרת מערך של אובייקטים מסוג Asset שמייצגים את הנתונים מהשאילתא.
fromXML	מייצרת אובייקט של Asset בהינתן Asset בפורמט XML.
toObjects	מחזירה מערך של אובייקטים מסוג Object כאשר כל אחד מייצג פרמטר אחד של ה Asset. זה משמש לצורך עדכון בסיס הנתונים.
getId/setId	מחזירה/מעדכנת את המספר המזהה של הנכס במערכת.
getRooms/setRooms	מחזירה/מעדכנת את מספר החדרים בנכס
getPrice/setPrice	מחזירה/מעדכנת את מחיר הנכס.
	ישנן מתודות נוספות לצורך קבלה ועדכון.

מחלקת ה AssetDBEnum

מחלקה זו מקשרת בין מחלקת Asset לבין הרשומה בבסיס הנתונים. המחלקה מכילה את שמות השדות המייצגים Asset בבסיס הנתונים. המחלקה יורשת ממחלקת BaseValuedEnum.

מחלקת ה AssetImpl

מחלקה זו מטפלת בבקשות של שליפה, הוספה, עדכון ומחיקה של נכסים בבסיס הנתונים ובנוסף, המחלקה מטפלת בחיפוש של נכסים במאגר הנתונים. הבקשה מועברת מ XmlProcessor Servlet והמחלקה מנהלת את הרישום ואת הקריאה מול בסיס הנתונים.

מתודות	הסבר
processRequest	נקראת מ XmlProcessor ובודקת באיזה פעולה מדובר: קבלת פרטי נכס קיים, יצירה, עדכון או מחיקת נכס, חיפוש נכסים במאגר. בהתאם לכל אחת מהפעולות, תקראנה המתודות המתאימות שמפורטות להלן, ותוחזר תשובה בפורמט XML.
getObjectXml	מקבלת את ה "id" של הנכס כפרמטר קוראת ל getObject לצורך קבלת ה Asset שמייצג את ה "id" הנ"ל במאגר הנתונים ומחזירה מחרוזת בפורמט XML המייצגת את הנכס הנ"ל.
getObject	מקבלת את ה "id" של הנכס כפרמטר ומחזירה אובייקט מסוג Asset מייצגת את הנכס הנ"ל.
deleteAsset	מוחקת מטבלת assets את הנכס לפי ה "id" שקיבלה כפרמטר.
createAsset	יוצרת נכס חדש בטבלת assets.
updateAsset	מעדכנת נכס קיים בטבלת assets.
getSearchResult	מבצעת חיפוש על טבלת assets בהתאם לחיתוכים שהמשתמש בחר במסך החיפוש. החיתוכים מיוצגים בעזרת מחלקת AssetSearchCriteria. המתודה מחזירה תשובה בפורמט XML עם פרטי הנכסים שנמצאו.

מחלקת ה AssetSearchCriteria

מחלקה זו משמשת כמיכל של קריטריוני החיפוש שהמשתמש בחר במסך החיפוש. אובייקט מסוג זה נוצר תוך עיבוד של עמודי ה JSP המטפלים בחיפוש נכסים, הופך לפורמט XML, מועבר ל XmlProcessor Servlet ואז מועבר למתודה getSearchResult במחלקת AssetImpl לצורך ביצוע החיפוש.

חבילת ה Data

חבילה זו מכילה מספר מחלקות אשר מטפלות במידע סטטי שנמצא בבסיס הנתונים ומשמש את הטפסים בעמודי ה JSP לצורך מילוי שדות הטופס ע"י המשתמש.

מחלקת ה ObjectData

מחלקה זו מייצגת רכיב מידע סטטי בטבלת object_data בבסיס הנתונים ומספקת מתודות לקבלת פרמטרים של רכיב המידע: סוג רכיב המידע וכינויו. המחלקה בשימוש ע"י מחלקת ObjectDataImpl שתואר בהמשך. המחלקה יורשת ממחלקת BaseForm. רכיב מידע יכול להיות מהסוגים הבאים: סוג נכס, פעולה (מכירה/השכרה), מספר חדרים, מין המשתמש, סוג הקרקע בה הנכס ממוקם ועוד.

מתודות	הסבר
process	בשל הירושה מ BaseForm מממשת את המתודה הזו אשר מקבלת תוצאה של שאילתא בבסיס הנתונים, ומייצרת מערך של אובייקטים מסוג ObjectData שמייצגים את הנתונים מהשאילתא.
getLabel/setLabel	מחזירה/מעדכנת את המספר תווית רכיב המידע.
getType/setType	מחזירה/מעדכנת את סוג רכיב המידע.

מחלקת ה ObjectTypesDBEnum

מחלקה זו מקשרת בין מחלקת ObjectData לבין הרשומה בבסיס הנתונים. המחלקה מכילה את שמות השדות המייצגים ObjectData בבסיס הנתונים. המחלקה יורשת ממחלקת BaseValuedEnum.

מחלקת ה ObjectDataImpl

המחלקה מטפלת בקריאה של רכיבי מידע סטטיים מטבלת object_data. הבקשה מועברת מ XmlProcessor Servlet. קריאת הנתונים מתבצעת מול CacheManager אשר טוען את המידע מטבלת object_data בזמן העליה, על מנת לחסוך קריאות ממבנה הנתונים.

מתודות	הסבר
handleObjectType	נקראת מ XmlProcessor ובודקת באיזה פעולה מדובר: קבלת כל רכיבי המידע הסטטיים, או קבלת רכיב מידע בודד. בהתאם לכל אחת מהפעולות, תקרנה המתודות המתאימות שמפורטות להלן, ותוחזר תשובה בפורמט XML.
getAllObjectsDataXML	מחזירה מחרוזת XML של כל רכיבי המידע הסטטי בעזרת קריאה ל getObjectsDataXML.
getObjectsDataXML	מחזירה רכיב מידע סטטי אחד כמחרוזת XML.

חבילת ה Address

חבילה זו מכילה מספר מחלקות אשר מטפלות במידע סטטי שנמצא בבסיס הנתונים ומשמש את הטפסים בעמודי ה JSP לצורך מילוי שדות הטופס ע"י המשתמש.

מחלקת ה City

מחלקה זו מייצגת עיר בטבלת cities בבסיס הנתונים ומספקת מתודות לקבלת פרמטרים של רכיב המידע: שם העיר והמזהה שלה במערכת. המחלקה בשימוש ע"י מחלקת AddressImpl שתואר בהמשך. המחלקה יורשת ממחלקת BaseForm.

מתודות	הסבר
process	בשל הירושה מ BaseForm מממשת את המתודה הזו אשר מקבלת תוצאה של שאילתא בבסיס הנתונים, ומייצרת מערך של אובייקטים מסוג City שמייצגים את הנתונים מהשאילתא.
getId/setId	מחזירה/מעדכנת את מזהה עיר.
getObjName/setObjName	מחזירה/מעדכנת את שם העיר.

מחלקת ה Street

מחלקה זו מייצגת רחוב בטבלת streets בבסיס הנתונים ומספקת מתודות לקבלת פרמטרים של רכיב המידע: שם הרחוב והמזהה שלו במערכת. המחלקה בשימוש ע"י מחלקת AddressImpl שתואר בהמשך. המחלקה יורשת ממחלקת BaseForm.

מתודות	הסבר
process	בשל הירושה מ BaseForm מממשת את המתודה הזו אשר מקבלת תוצאה של שאילתא במבנה הנתונים, ומייצרת מערך של אובייקטים מסוג Street שמייצגים את הנתונים מהשאילתא.
getId/setId	מחזירה/מעדכנת את מזהה הרחוב.
getObjName/setObjName	מחזירה/מעדכנת את שם הרחוב.

מחלקת ה AddressImpl

המחלקה מטפלת בקריאה של רכיבי מידע סטטיים מטבלאות Cities ו Streets. הבקשה מועברת מ XmlProcessor Servlet.

מטודות	הסבר
processCityRequest	נקראת מ XmlProcessor ובודקת באיזה פעולה מדובר: קבלת כל הערים, או קבלת ערים ששמן מכיל בפרמטר שהתקבל. בהתאם לכל אחת מהפעולות, תקרנה המתודות המתאימות שמפורטות להלן, ותוחזר תשובה בפורמט XML.
getAllCities	מחזירה מחרוזת XML של כל הערים שנמצאות בטבלת cities. המחרוזת מתקבלת בעזרת העברת תוצאות החיפוש למתודה makeXML שבונה את התשובה.
getApproximateCities	מחזירה מחרוזת XML של כל הערים בטבלת cities ששמן מכיל את המחרוזת שנתקבלה כפרמטר. תוצאת החיפוש נבנית בעזרת המתודה makeXML.
processStreetRequest	נקראת מ XmlProcessor ובודקת באיזה פעולה מדובר: קבלת כל הרחובות, או קבלת רחובות ששמן מכיל בפרמטר שהתקבל. בהתאם לכל אחת מהפעולות, תקרנה המתודות המתאימות שמפורטות להלן, ותוחזר תשובה בפורמט XML.
getAllStreets	מחזירה מחרוזת XML של כל הרחובות שנמצאים בטבלת streets. המחרוזת מתקבלת בעזרת העברת תוצאות החיפוש למתודה makeXML שבונה את התשובה.
getApproximateStreets	מחזירה מחרוזת XML של כל הרחובות בטבלת streets ששמן מכיל את המחרוזת שנתקבלה כפרמטר. תוצאת החיפוש נבנית בעזרת המתודה makeXML.
makeXML	מקבלת את תוצאות החיפוש כמערך של אובייקטים מסוג BaseForm ובונה מהן מחרוזת XML.

חבילת ה XML

חבילה זו מאפשרת עיבודים על XML ומכילה מחלקה אחת.

מחלקת ה DomWrapper

מחלקה זו מספקת מעטפת לטיפול ב XML, ומספקת ממשקים שמאפשרים גישה ועדכון פרמטרים השונים ב XML. המחלקה היא מחלקה המשמשת את המחלקות השונות שמתממשקות מול מחלקות ה Servlets ואת מחלקות ה Servlets עצמן.

חבילת ה Exceptions

חבילה זו מכילה שתי מחלקות אשר משמשות לזריקה של חריגות.

מחלקת ה *UserAlreadyExistException*

משמשת לזריקת חריגה שמייצגת משתמש שכבר קיים. בשימוש ממחלקת UserImpl כאשר מגדירים משתמש חדש.

מחלקת ה *UserNotFoundException*

משמשת לזריקת חריגה שמייצגת משתמש שלא נמצא. בשימוש ממחלקת UserImpl כאשר משתמש מבצע כניסה למערכת.

חבילת ה SQL

חבילה זו נותנת שתי מחלקות לעזרת שאילתות ה SQL המתבצעות במערכת.

מחלקת ה *AggregateResultProcessor*

מחלקה אשר מממשת ResultProcessor על מנת להחזיר תוצאות של שאילתות "צוברות" כגון count(*).

מחלקת ה DBUtils

מחלקה עם מתודה סטטית אחת בשם replaceNull שתפקידה להחליף אובייקטים NULL לאובייקטים NULL מסוג SQL, כך ששאילתות SQL ירוצו על אובייקטים שהם NULL, הן לא ייפלו.

חבילת ה Common

חבילה זו מכילה אוסף אל מחלקות אשר משרתות את כלל המחלקות בתוכנית.

מחלקת ה BaseForm

מחלקה אבסטרקטית, שכל המחלקות במערכת, אשר מייצגות רכיבים בבסיס הנתונים יורשים ממנה. המחלקה מממשת את הממשק ResultProcessor ובשל כך היא משמשת לצורך עיבוד של תשובות משאילתות SQL.

מחלקת ה CacheManager

המחלקה מחזיקה HashMap בזיכרון של כל רכיבי המידע הסטטיים שנמצאים בטבלת object_data בכדי לחסוך קריאות רבות וחוזרות של הנתונים אשר במילא לא משתנים. המתודה initObjectsData נקראת ע"י המחלקה StartupServlet בזמן עליית שרת ה Tomcat.

מחלקת ה MinMax

מחלקה זו מייצגת נתוני מינימום ומקסימום ומשרתת את מנגנון החיפוש לצורך העברת ערכי מינימום ומקסימום של שדות שונים שהמשתמש בחר במסך החיפוש.

מחלקת ה Sort

משמשת לצורך שמירת בקשת המיון: לפי איזה עמודה למיין, והאם למיין בסדר עולה או בסדר יורד. המחלקה בשימוש מנגנון החיפוש לצורך העברת נתוני המיון אל השרת.

מחלקת ה Utils

מחלקה זו מכילה מספר מתודות סטטיות לצרכים שונים.

מתודות	הסבר
getDateFromString	מתרגמת מחרוזת שמייצגת תאריך, לאובייקט מסוג Date.
getStringFromDate	מתרגמת אובייקט מסוג Date למחרוזת שמייצגת את התאריך.
manipulateRequest	מתרגמת את בקשת ה HTTP לבקשת XML תוך תרגום התווים בבקשה לפורמט של UTF-8 בעזרת מתודת decodeString
decodeString	מתרגמת מחרוזת לפורמט של UTF-8.